Self-Pruning Neural Network on CIFAR-10

Technical Case Study Report

Abhay Tiwari (RA2311003010662)

SRM Institute of Science and Technology, Kattankulathur

1. Introduction

Modern deep neural networks are heavily over-parameterized. A model trained on CIFAR-10 can easily have millions of parameters, most of which contribute very little to the final prediction. Pruning is a well-studied technique to reduce this redundancy — the idea is to identify and remove weights that don't carry meaningful information, without hurting accuracy significantly.

The traditional approach is to train the model fully first, then prune the low-magnitude weights, and fine-tune. This works, but has a fundamental inefficiency: you're spending compute on training weights that you'll eventually throw away anyway. Dynamic pruning, where the model learns what to prune during training itself, is more interesting from a systems perspective. The network figures out which connections are worth keeping as it's learning, and allocates capacity more efficiently. For real-world deployment — especially on edge devices or in resource-constrained environments — this kind of adaptive compression is genuinely useful.

2. Problem Understanding

Post-training pruning has a core limitation: the model was never trained with the expectation of losing those weights. When you remove them after the fact, the remaining weights haven't been optimized to compensate. You need an additional fine-tuning stage, and even then, the model often doesn't recover fully if you push sparsity too high.

The more principled approach is to make pruning part of the learning objective. If the model knows — during training — that certain connections might get switched off, the active weights adjust accordingly. They learn to be more robust, to not depend on weights that might be removed. This is fundamentally different from post-hoc removal because the optimization landscape itself changes.

Essentially, instead of asking "which weights are small enough to delete?" after the fact, we're asking "which weights does this network actually need to solve the task?" during training. That's a much better-posed question.

3. Approach and Design

The core idea is straightforward: attach a learnable scalar gate parameter to each weight in the prunable layers. Instead of directly zeroing out weights, we compute a soft gate value using the sigmoid function applied to the gate score, and multiply the weight by this gate. This gives us a differentiable proxy for pruning.

Sigmoid is the right choice here because it maps any real number to (0, 1). A gate score of, say, 5 gives a gate value close to 1 (weight mostly active), while a score of −5 gives something close to 0 (weight effectively pruned). During training, the gate scores are updated via backpropagation along with the actual weights, so the model learns jointly which connections to keep and which to drop.

One important design choice is initialization. I initialized the gate scores to 3.0, which through sigmoid gives a starting gate value of about 0.95 — close to 1. The reasoning: you don't want the model to start training with half its weights already suppressed. Let it start with all connections active, and let the sparsity loss gradually push less useful gates toward zero. Starting too low would destabilize early training; starting too high (e.g., 10.0) would make the gates too slow to move initially. A value of 3.0 gives a good balance — active enough to not interfere with early learning, but not so large that the gates become unresponsive.

4. Loss Function and Reasoning

The total training loss is a combination of two terms: CrossEntropy loss for classification, and an L1 sparsity penalty on the gate values.

CrossEntropy is the standard choice for multi-class classification — no surprises there. The interesting part is the sparsity term. I chose L1 regularization on the gate values (after sigmoid) rather than L2. The reason is geometric: L1 pushes values toward exactly zero, which is what we want for hard pruning. L2 keeps values small but nonzero, meaning weights are suppressed but not actually removed. For our purpose — achieving real sparsity — L1 is the correct choice.

Normalization of the sparsity loss also matters for training stability. The raw sum of gate values scales with the number of parameters in the prunable layers, which can be in the thousands. Without normalization, even a small lambda would create a massive gradient signal relative to the CrossEntropy loss. I normalized by dividing by the total number of gate parameters, which keeps the sparsity term's magnitude stable regardless of model size.

5. Training Strategy

I used the Adam optimizer with separate learning rates for the model weights and the gate parameters. The weights were updated with a standard learning rate of 1e-3. The gate parameters got a higher learning rate of 1e-2, roughly 10× faster.

The reasoning: gate parameters are scalars that need to move from their initial values (near 3.0) down toward negative values if they're going to be pruned. With the same learning rate as the weights, this movement would be very slow. Using a higher learning rate for the gates lets them respond more quickly to the sparsity loss signal, without disrupting the actual weight learning.

I also applied gradient clipping (max norm = 1.0) to avoid instability from large gate gradients early in training, when the loss landscape is still settling. It's a relatively minor but useful safeguard.

6. Experimental Setup

Component	Details
Dataset	CIFAR-10 (60,000 images, 10 classes, 32×32 RGB)
Model	CNN backbone + Prunable fully connected layers with learnable gates
Optimizer	Adam (weights: lr = 1e-3, gate params: lr = 1e-2)
Epochs	30

Component	Details
Batch Size	128
Pruning Target	Fully connected layers (gated)
Sparsity Loss	L1 on sigmoid(gate scores), normalized by parameter count

7. Results

Lambda (λ)	Test Accuracy	Sparsity
0.5	81.06%	69.37%
2.0	79.97%	82.48%
8.0	79.05%	94.25%

8. Analysis and Insights

The results reflect a clean trade-off between sparsity and accuracy, which is exactly what you'd expect from this kind of regularization approach.

Lambda controls how much the sparsity loss is weighted relative to the classification loss. At λ = 0.5, the model is mostly focused on accuracy, and pruning is conservative — 69% sparsity with 81% accuracy. As lambda increases, the optimizer is pushed harder to eliminate unnecessary gates, and sparsity climbs significantly. At λ = 8.0, over 94% of the FC layer gates are effectively zero, but accuracy only drops to about 79%.

That's a pretty solid result — losing less than 2.1% accuracy in exchange for pruning out 94% of the parameters in the dense layers is meaningful compression. The reason accuracy doesn't collapse at high sparsity is that the remaining 5–6% of active weights are the ones that actually matter for the task. The model is forced to concentrate its capacity where it's needed.

Lambda = 2.0 stands out as the practical sweet spot: 79.97% accuracy with 82.48% sparsity. The accuracy loss relative to λ = 0.5 is just over 1%, but you're getting 13 percentage points more pruning. For any real deployment scenario where memory and inference cost matter, this is the better operating point.

One observation worth calling out: the final classification layer tends to retain more active gates than the earlier dense layers. This makes sense — the final layer maps the learned representation directly to class scores. Every output neuron here corresponds to a class, so the connections going into it directly determine what gets predicted. Pruning aggressively in the final layer is risky because there's no downstream layer to compensate. Earlier layers have more representational redundancy, so they can afford higher sparsity without losing classification-relevant information.

9. Visualization Insights

After training, I plotted histograms of the gate values (post-sigmoid) across all prunable layers. The distribution is clearly bimodal: a large spike near 0 and a smaller cluster near 1, with relatively little in between.

The spike near 0 represents gates that have been effectively pruned — the sigmoid output is so close to zero that the corresponding weight is for all practical purposes inactive. The cluster near 1 represents connections the model decided to keep.

The bimodal distribution is a good sign: it means the gating mechanism isn't wishy-washy. The model is making relatively clear decisions about what to keep and what to remove, rather than leaving everything in some ambiguous intermediate state. A flat distribution would suggest the gating isn't working well; a sharp bimodal distribution indicates the training objective is actually pushing gates toward meaningful hard pruning.

10. Conclusion

This project demonstrates that it's feasible to build a network that learns to prune itself during training, without requiring a separate post-training compression step. On CIFAR-10, using learnable sigmoid-gated weights with L1 sparsity regularization, the model achieves up to 94% sparsity in the fully connected layers while retaining competitive classification accuracy. The lambda parameter provides a clean, interpretable knob to trade accuracy for sparsity depending on deployment constraints.

The key insight is that when sparsity is part of the loss during training, the surviving weights are better calibrated — they've been trained under the pressure of having to compensate for their pruned neighbors. This makes the final compressed model more reliable than one obtained by post-hoc pruning at the same sparsity level. This approach is practical for real-world deployment where memory and compute constraints are critical.

11. Future Work

There are a few natural extensions worth exploring:

1. Extending gated pruning to convolutional layers. The current implementation targets fully connected layers, but CNNs derive most of their parameters and compute cost from conv layers. Applying learnable gates channel-wise or filter-wise there would make the compression substantially more impactful.

2. Structured pruning over unstructured pruning. Individual weight removal (unstructured) requires sparse matrix support to actually speed up inference on standard hardware. Structured pruning — removing entire neurons or channels — gives real latency improvements on dense hardware without specialized kernels.

3. Scaling to larger datasets like CIFAR-100 or ImageNet. The sparsity-accuracy trade-off likely behaves differently at scale. Larger datasets provide richer signal, and it's an open question whether the same gating approach remains effective at the level of ResNets or EfficientNets.

End of Report